Requirements Document

Campus Data Orchestrator — An MCP-Based Campus Data Orchestrator

Field	Detail
Course	BIL 481 Software Engineering, Fall 2026, TOBB University of Economics and Technology
Assignment	Project Assignment 1 (SALT Documentation / No Code Submission)
Group Name	Group-ASYM
Group Members	Murat Can Işık, Arda Çelik, Yunus Emre Biçer, Ahmet Sefa Efendioğlu
Document Lead & Primary Author	**Murat Can Işık** (Requirements Lead & Full-Stack Developer)
Quality Reviewer & Release Lead	**Ahmet Sefa Efendioğlu** (QA Lead & Cross-Audit Verification)
Document Version / Date	Version 1.0 (Final Deliverable), 1 November 2026
Repository Filepath	`docs/Requirements_Document.md`
Related Documents	`docs/Concept_Notes.pdf`, `Project_Definition_Document.md`, `Project_Plan_Document.md`

Document Authorship & Team Attribution

As required by the BIL 481 Project Assignment 1 specification, each document must identify all contributors, their project roles, and their specific responsibilities in producing this deliverable:

Member Name	Student ID	GitHub Account	Primary Project Role	Contribution to this Document
Murat Can Işık	241101013	`@cmuratt`	Requirements Engineer & Lead Developer	**Primary Author & Sole Owner:** Authored the entire document from scratch; drafted sections 1–6 (FR-01 to FR-20, NFR-01 to NFR-09), defined acceptance criteria, I/O schemas, and traceability tables.
Ahmet Sefa Efendioğlu	241101066	`@Sefa-Efendioglu`	QA & Release Lead	**Cross-Audit & Verification:** Performed guideline compliance checks (10+ FR, 5+ NFR check), verified document-specific task matrix, audited Table of Contents, and verified PDF compile pipeline.
Arda Çelik	241101065	`@ardacelik11`	Project Definition Lead	**Scope Synchronization:** Verified alignment between Project Definition Key Features (F-1 to F-7) and functional requirement declarations.
Yunus Emre Biçer	241101009	`@YunusEmre113`	Project Planner & Architect	**Schedule & Effort Verification:** Validated the 3.0-hour document creation effort and mapped requirements against the 70-hour project budget.

Document-Specific Task Matrix

The task matrix below outlines the isolated work packages executed to research, specify, review, and finalize this Requirements Document for Assignment 1:

Task ID	Task Description	Assignee	Reviewer	Allocated Effort	Status
TSK-REQ-01	Analysis of Concept Note baseline (FR-1 to FR-6, NFR-1 to NFR-4) and system pseudocode contracts	Murat Can Işık	Ahmet Sefa Efendioğlu	0.50 h	Completed
TSK-REQ-02	Formalization of system definitions, user classes, operational assumptions, and verification conventions	Murat Can Işık	Arda Çelik	0.40 h	Completed
TSK-REQ-03	Specification of Functional Requirements FR-01 to FR-12 (MCP Server, Tools, Gateway, Data Resilience)	Murat Can Işık	Yunus Emre Biçer	0.85 h	Completed
TSK-REQ-04	Specification of Functional Requirements FR-13 to FR-20 (AI Agent, Client Views, UX Error Handling, Audit)	Murat Can Işık	Arda Çelik	0.50 h	Completed
TSK-REQ-05	Formulation of Non-Functional Requirements NFR-01 to NFR-09 with measurable metrics	Murat Can Işık	Ahmet Sefa Efendioğlu	0.25 h	Completed
TSK-REQ-06	Construction of Requirements Traceability Matrix (RTM) and Concept Note cross-mapping	Murat Can Işık	Yunus Emre Biçer	0.20 h	Completed
TSK-REQ-07	Quality assurance checklist audit, Markdown formatting validation, and PDF conversion review	Ahmet Sefa Efendioğlu	Murat Can Işık	0.30 h	Completed
Total Effort	**Collaborative Authoring & Review Effort**			**3.00 h**	**Completed**

Note on Effort Accounting: Murat Can Işık spent 2.70 hours on direct document drafting, and Ahmet Sefa Efendioğlu spent 0.30 hours on formal QA review. Both efforts fall under the 10.0-hour "Analysis & Plan" milestone (19 Oct – 1 Nov) defined in the project plan.

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 System Overview & Architecture Context
    - 1.3 Definitions, Acronyms, and Abbreviations
2. Users and Stakeholders
3. Assumptions and Dependencies
4. Requirement Conventions and Notation
5. Functional Requirements (FR)
    - 5.1 Core MCP Server and Tool Registry

        - FR-01: MCP Protocol Compliance and Strict Schema Validation
        - FR-02: Transport Layer Support (Streamable HTTP and Stdio)
        - FR-03: Cafeteria Daily Menu Retrieval Service
        - FR-04: Campus Shuttle Ring Schedule and Departures
        - FR-05: Real-Time Library Occupancy Tracking
        - FR-06: Student Information System (SIS) Course Timetable
    - 5.2 API Gateway, Perimeter Security, and Identity
        - FR-07: Authentication Verification and Perimeter Token Termination
        - FR-08: Per-Identity Tiered Rate Limiting
    - 5.3 Data Resilience, Caching, and Connector Architecture
        - FR-09: Multi-Tiered Redis Caching with Granular TTLs
        - FR-10: Automated Stale-If-Error Fallback Mechanism
        - FR-11: Connector-Level Isolated Circuit Breaking
        - FR-12: Unified Connector Interface and Hot-Swappable Mock Modes
    - 5.4 AI Agent and Orchestration Intelligence
        - FR-13: Autonomous Multi-Tool Planning and Contextual Synthesis
        - FR-14: Zero-Leakage PII Scrubbing in Agent Prompts
    - 5.5 Web Client Presentation and UX Interaction
        - FR-15: Authentication State and Session Profile Management
        - FR-16: Client-Side Bilingual Deterministic Intent Router
        - FR-17: Conversational Assistant View with SSE Streaming & Telemetry Badges
        - FR-18: Fast-Access Campus at a Glance Dashboard
        - FR-19: Comprehensive Visual Error, Latency, and SSE Reconnection Handling
        - FR-20: PostgreSQL Telemetry Auditing and Latency Logging
6. Non-Functional Requirements (NFR)
    - NFR-01: Performance — Layered Latency Bounds and Decoupled Inference
    - NFR-02: Efficiency — Redis Cache Hit Ratio
    - NFR-03: Reliability — Fault Isolation and Cascading Failure Prevention
    - NFR-04: Scalability — Stateless Concurrency and Load Distribution
    - NFR-05: Security — Perimeter Token Isolation and Public TLS
    - NFR-06: Usability — Navigational Efficiency and Usability Standards
    - NFR-07: Portability & Responsive Viewport Compatibility
    - NFR-08: Maintainability & Offline Test Coverage
    - NFR-09: Deployability — Single-Command Container Orchestration
7. Requirements Traceability Matrix (RTM)
8. Revision History

---

1. Introduction

1.1 Purpose

This document establishes the comprehensive functional and non-functional requirements for the **Campus Data Orchestrator** project. It serves as the baseline contract for system design, implementation, and subsequent test suites in BIL 481 (Fall 2026). Each requirement is uniquely identified, prioritized via the MoSCoW methodology, specified with explicit inputs, processes, and outputs, and paired with verifiable acceptance criteria.

1.2 System Overview & Architecture Context

Campus operations in modern universities are digitally fragmented: daily cafeteria menus are published on catering CMS portals, lecture schedules reside inside proprietary Student Information Systems (SIS), library seat utilization is tracked across turnstile hardware feeds, and campus shuttle rings are monitored through third-party GPS systems.

The Campus Data Orchestrator bridges these silos using the open-standard **Model Context Protocol (MCP)** over JSON-RPC 2.0. The system decouples presentation, reasoning, and data integration across five primary architectural tiers:

1. **Presentation Tier (Next.js):** Hosts the dual-view interface (Conversational Assistant and Live Campus Glance Dashboard) accompanied by a lightweight deterministic intent router.
2. **API Gateway Tier (Fastify):** Enforces zero-trust perimeter security, terminates OAuth2/JWT tokens, scrubs claims, and executes per-student rate limiting.
3. **AI Agent Tier (MCP Client SDK):** Orchestrates multi-step reasoning plans, invokes MCP tools via standardized schemas, and synthesizes answers without retaining student credentials.
4. **Middleware Tier (MCP Server):** A stateless Node.js engine executing tool dispatching, Redis caching, circuit breaking, and PostgreSQL telemetry logging.
5. **Connector Tier:** Pluggable integration adapters paired with deterministic synthetic test stubs for air-gapped CI/CD execution.

1.3 Definitions, Acronyms, and Abbreviations

Term	Formal Definition
MCP	Model Context Protocol; an open architectural protocol utilizing JSON-RPC 2.0 to expose tools and contextual data to AI agents.
JSON-RPC 2.0	A stateless, lightweight remote procedure call (RPC) protocol specifying structured JSON request and response envelopes.
JWT	JSON Web Token; a compact, URL-safe means of representing claims securely between two parties.
SSE	Server-Sent Events; an HTTP standard enabling unidirectional real-time data streaming from server to client.
TTL	Time-To-Live; the lifespan in seconds that an item resides in Redis before being marked expired.
Stale-If-Error	A cache fault-tolerance strategy where expired data is served with an `"isStale": true` flag if upstream data retrieval fails.
Circuit Breaker	A resilience pattern that arrests outbound requests to an unhealthy service once failure thresholds are tripped.

Term	Formal Definition
PII	Personally Identifiable Information; data fields that can directly identify an individual (e.g., student name, ID number, email).
P95	95th Percentile; the statistical threshold under which 95% of measured latency observations fall.
SUS	System Usability Scale; a standardized, reliable 10-item Likert scale tool for measuring the usability of software systems.
SIS	Student Information System; university database storing student course registrations and academic timetables.

2. Users and Stakeholders

User Class	Description	Scope of Interaction	Relevant Requirements
Authenticated Student	Enrolled university student holding a valid SSO identity token.	Accesses public tools and private academic timetable data (`get_course_schedule`). Uses conversational assistant and personalized dashboard.	All FRs and NFRs
Guest User (Faculty / Visitor)	Unauthenticated user accessing the platform without SSO credentials.	Can access only public tools (`cafeteria` , `shuttle` , `library`). Blocked from invoking private student endpoints.	FR-01 to FR-05, FR-08 to FR-12, FR-16 to FR-19
MCP Client Developer	External software engineer integrating an external AI agent or IDE via MCP.	Interacts directly with the MCP server over Streamable HTTP or standard I/O using MCP specification standards.	FR-01, FR-02, NFR-01, NFR-04
System Administrator / Course Staff	Course instructor, teaching assistant, or operations maintainer evaluating quality.	Evaluates system uptime, verifies telemetry logs, measures cache ratios, and inspects mock-mode pipelines.	FR-20, NFR-01 to NFR-09

3. Assumptions and Dependencies

- **A-1 (Mock Data Dependency):** Production campus database credentials and live university turnstile hardware are inaccessible to students. Therefore, primary verification relies on **synthetic test stubs** providing deterministic mock data. Connectors implement live scraping as an experimental extension only.
- **A-2 (Identity Token Baseline):** Authentication is verified against a local mock SSO token issuer generating RSA-signed JWTs containing the standard claims `sub` (Student ID), `name` , `email` , `role` , and `exp` .
- **A-3 (LLM Provider Key Handling):** AI Agent orchestration utilizes the Google Gemini API. The API key resides strictly in local `.env` files (excluded via `.gitignore`) and is never committed to GitHub or leaked to client browsers.
- **A-4 (Standardized Benchmark Environment):** All numeric performance targets (sub-250ms P95, 100 concurrent requests) are calibrated for local multi-container execution inside Docker Compose on standard developer hardware (Apple Silicon / x86_64, minimum 16GB RAM) with synthetic upstream connector delays simulated at 100–300 ms.

- **A-5 (Timezone and Locale Standards):** All scheduling calculations assume Türkiye Time (UTC+3). All exchange dates conform strictly to ISO-8601 (`YYYY-MM-DD`).

---

4. Requirement Conventions and Notation

Each requirement in this specification is documented according to the following conventions:

- **Identifier:** `FR-xx` for Functional Requirements; `NFR-xx` for Non-Functional Requirements.
- **Priority (MoSCoW):**
  - **Must:** Absolute baseline requirement; necessary for Assignment 1 deliverable completeness.
  - **Should:** Highly recommended; planned for core delivery unless severe timeline blockers emerge.
  - **Could:** Optional extension or advanced enhancement (e.g., internal service mTLS).
- **Verification Methods:**
  - **T (Automated Test):** Unit, integration, or schema test executable in automated CI pipelines (Jest/Supertest).
  - **D (Demonstration):** Observable operational workflow in the user interface or MCP Inspector.
  - **I (Inspection):** Manual code review, database audit, or network packet inspection.
  - **M (Measurement):** Quantitative metric captured via load generators (K6/Autocannon) or telemetry logs.

---

5. Functional Requirements (FR)

5.1 Core MCP Server and Tool Registry

FR-01: MCP Protocol Compliance and Strict Schema Validation

- **Priority:** Must | **Verification:** T, D
- **Description:** The core middleware server shall implement the official Model Context Protocol (MCP) specification over JSON-RPC 2.0. It shall register exactly four tools: `get_cafeteria_menu`, `get_shuttle_schedule`, `get_library_occupancy`, and `get_course_schedule`.
- **Inputs:** JSON-RPC 2.0 discovery requests (`tools/list`) and invocation requests (`tools/call`) containing tool name and argument JSON objects.
- **Processing:** The server registers each tool with a type-safe JSON Schema specifying property types, mandatory parameters, and field descriptions. Upon receiving a call, the incoming payload is validated against the schema before dispatching to connector handlers.
- **Outputs:** MCP standard JSON envelope containing structured result blocks (`MCPToolResponse`) or standardized JSON-RPC error codes.
- **Acceptance Criteria:**
  1. A `tools/list` request returns all four registered tools with valid JSON Schema definitions.
  2. Submitting a tool invocation with invalid parameters (e.g., an invalid date format or string instead of integer) fails validation and immediately returns JSON-RPC error code `-32602 (Invalid params)` without triggering upstream connector calls.

FR-02: Transport Layer Support (Streamable HTTP and Stdio)

- **Priority:** Must (Streamable HTTP), Should (stdio) | **Verification:** T, D
- **Description:** The MCP server shall be accessible over Streamable HTTP (enabling Server-Sent Events for client-server bidirectional flows) and standard input/output (`stdio`) for local MCP clients.
- **Inputs:** HTTP POST connections to `/mcp` or input pipes over process `process.stdin` .
- **Processing:** The server inspects the transport interface and binds JSON-RPC listeners to the active stream.
- **Outputs:** Piped standard output or chunked HTTP SSE stream.
- **Acceptance Criteria:** The official MCP Inspector GUI can successfully attach to the server over Streamable HTTP, list all registered tools, and execute test invocations.

FR-03: Cafeteria Daily Menu Retrieval Service

- **Priority:** Must | **Verification:** T
- **Description:** The server shall provide the tool `get_cafeteria_menu` to return daily cafeteria nutritional details.
- **Inputs:** `date` (String, ISO `YYYY-MM-DD` , optional; defaults to today's date).
- **Processing:** Validates date format. Checks Redis for key `cache:dining:<date>` . On a cache miss, calls the cafeteria connector to fetch/parse the menu, caches the result for 3600 seconds, and formats the output.
- **Outputs:** JSON object containing:
  - `lunch` : `{ soup: string, mainDish: string, vegOption: string, sideDish: string, calories: number, allergens: string[] }`
  - `dinner` : `{ soup: string, mainDish: string, sideDish: string, calories: number }`
  - `available` : `true` (or `false` with a descriptive string for weekends/holidays).
- **Acceptance Criteria:** Querying a valid weekday returns both lunch and dinner fields with calorie totals > 0. Querying a weekend returns `"available": false` with a clear explanation without throwing an unhandled exception.

FR-04: Campus Shuttle Ring Schedule and Departures

- **Priority:** Must | **Verification:** T
- **Description:** The server shall provide the tool `get_shuttle_schedule` to return live departures and upcoming timetable slots.
- **Inputs:** `route` (Enum: `MAIN_RING` , `METRO_LINE` , `DORMITORY_LINE` ; defaults to `MAIN_RING`).
- **Processing:** Validates route enum. Evaluates scheduled timetable departures against current system time (UTC+3), discards past departures, checks active GPS fleet counts, and caches data in Redis for 120 seconds.
- **Outputs:** JSON object containing:
  - `routeId` (String), `routeName` (String).
  - `nextDepartures` : Chronologically sorted array of departure strings (e.g., `["12:35", "12:50", "13:05"]`).
  - `activeVehicles` (Integer), `lastUpdated` (ISO timestamp).
- **Acceptance Criteria:** All times in `nextDepartures` must be strictly later than the request execution timestamp. Submitting an invalid route name returns an invalid-params error listing valid enum options.

FR-05: Real-Time Library Occupancy Tracking

- **Priority:** Must | **Verification:** T

- **Description:** The server shall provide the tool `get_library_occupancy` to expose real-time seat availability across library study floors.
- **Inputs:** `floor` (Enum: `FLOOR_1`, `FLOOR_2`, `FLOOR_3`, or `ALL`; defaults to `ALL`).
- **Processing:** Queries the library IoT connector. If a specific floor is requested, isolates that floor; otherwise, aggregates total building metrics. Caches data in Redis for 60 seconds.
- **Outputs:** JSON object containing:
  - `timestamp` (ISO string), `isSimulated` (Boolean).
  - `totalCapacity` (Integer), `totalOccupied` (Integer).
  - `floors`: Array of `{ floorId, name, occupiedSeats, maxSeats, isQuietArea }`.
- **Acceptance Criteria:** 0 ≤ occupiedSeats ≤ maxSeats for all floors. The sum of per-floor `occupiedSeats` equals `totalOccupied`. In mock mode, `isSimulated` must be `true`.

FR-06: Student Information System (SIS) Course Timetable

- **Priority:** Must | **Verification:** T
- **Description:** The server shall provide the tool `get_course_schedule` to return personal daily schedules for authenticated students.
- **Inputs:** `day` (String: `TODAY`, `MONDAY`, `TUESDAY`, etc., defaults to `TODAY`) and injected internal header `X-Authenticated-Student-Id`.
- **Processing:** The server inspects request context for a verified student ID. If absent, execution halts with an authorization error. When present, it retrieves the student's enrolled courses, filters by the specified day, and caches the result for 300 seconds per student.
- **Outputs:** Array of `{ courseCode, courseTitle, instructor, classroom, startTime, endTime }`.
- **Acceptance Criteria:**
  1. Requests executed without a verified student identity fail deterministically with an "Authentication required" error.
  2. Tool arguments attempting to pass arbitrary student IDs are rejected by JSON Schema enforcement (identity can only be sourced via gateway headers).
  3. Classes for the authenticated student are returned sorted chronologically by `startTime`.

5.2 API Gateway, Perimeter Security, and Identity

FR-07: Authentication Verification and Perimeter Token Termination

- **Priority:** Must | **Verification:** T, I
- **Description:** The Fastify API Gateway shall terminate external student authentication (OAuth2 / SSO JWT) at the perimeter.
- **Inputs:** Inbound HTTP header `Authorization: Bearer <JWT>`.
- **Processing:** The gateway verifies token signature, expiry, and issuer against public keys. If valid, the gateway strips the raw `Authorization` header, strips any spoofed inbound `X-Authenticated-*` headers, and injects verified internal headers: `X-Authenticated-Student-Id` and `X-Authenticated-User-Role`.
- **Outputs:** Sanitized internal HTTP request forwarded downstream, or HTTP 401 Unauthorized for invalid/expired tokens.

- **Acceptance Criteria:** Network packet inspection between Gateway, MCP Server, and LLM confirms a complete absence of raw JWT bearer tokens (100% token isolation). Tampered or expired tokens receive HTTP 401.

FR-08: Per-Identity Tiered Rate Limiting

- **Priority:** Must | **Verification:** T
- **Description:** The gateway shall enforce rate limits using Redis atomic counters to prevent denial-of-service and abuse.
- **Inputs:** Authenticated student ID (or client IP address for guest users).
- **Processing:** Tracks requests within a sliding 60-second window. The rate ceiling is set to 30 requests per minute per authenticated student, and 30 requests per minute per IP for guest routes.
- **Outputs:** Forwarded request, or HTTP 429 Too Many Requests with a `Retry-After` header.
- **Acceptance Criteria:** The 31st request within a 60-second window returns HTTP 429. Subsequent requests succeed once the rate window resets.

5.3 Data Resilience, Caching, and Connector Architecture

FR-09: Multi-Tiered Redis Caching with Granular TTLs

- **Priority:** Must | **Verification:** T
- **Description:** The MCP server shall cache tool results in Redis with fine-grained TTLs matching the volatility of underlying campus data feeds.
- **Inputs:** Cache key (`cache:<tool>:<params>`), computed payload, and TTL duration.
- **Processing:** Stores serialized JSON payloads in Redis with specific TTLs:
  - Cafeteria Menu: 3600 seconds (1 hour).
  - Shuttle Timetables: 120 seconds (2 minutes).
  - Library Occupancy: 60 seconds (1 minute).
  - Student Timetable: 300 seconds (5 minutes, isolated per student). Physical Redis retention is configured to 3 × TTL to maintain fallback data for stale recovery.
- **Outputs:** Cache hit payload served in < 15 ms, or fallback on cache miss.
- **Acceptance Criteria:** Repeated queries within the active TTL window do not increment upstream connector call counters and return cached payloads with `"isStale": false`.

FR-10: Automated Stale-If-Error Fallback Mechanism

- **Priority:** Must | **Verification:** T
- **Description:** If an upstream campus service times out or fails, the server shall serve the last cached record with an explicit staleness indicator.
- **Inputs:** Upstream network timeout (cutoff at 800 ms) or HTTP 5xx error.
- **Processing:** The server catches the exception, checks Redis for expired cached entries, and if found, returns the data flagged with `"isStale": true` alongside its original creation ISO timestamp. If no cache exists, it returns a structured JSON error.
- **Outputs:** JSON response: `{ isError: false, content: [{ type: "json", data: {...}, isStale: true, timestamp: "..." }] }`.

- **Acceptance Criteria:** In a test with an upstream cafeteria failure, the server returns the cached menu with `"isStale": true` in under 50 ms. With an empty cache, it returns a structured user-readable error within 1 second without crashing.

FR-11: Connector-Level Isolated Circuit Breaking

- **Priority:** Must | **Verification:** T
- **Description:** To prevent cascading failures, the MCP server shall implement an automated circuit breaker per connector.
- **Inputs:** Consecutive failure counts from individual campus connectors.
- **Processing:** If a connector experiences 3 consecutive failures (timeouts or errors), its circuit opens for a 60-second cooldown period. While open, calls to that connector immediately return stale cache or error responses without making network calls. After 60 seconds, one probe request is permitted (half-open state).
- **Outputs:** Immediate circuit-tripped fallback response during the open window.
- **Acceptance Criteria:** Inducing failures on the dining connector trips its circuit breaker after 3 calls. Subsequent dining queries make zero network attempts for 60 seconds, while concurrent calls to library and shuttle tools continue executing normally.

FR-12: Unified Connector Interface and Hot-Swappable Mock Modes

- **Priority:** Must | **Verification:** T, I
- **Description:** Every data connector shall implement the TypeScript interface `ICampusConnector<T>` and provide both live and deterministic synthetic mock modes.
- **Inputs:** Environment configuration flags (e.g., `CAFETERIA_CONNECTOR_MODE=mock|live`).
- **Processing:** Connectors switch between live scraping/APIs and synthetic generators based on environment configuration without code changes. Mock generators use deterministic seeds to produce reproducible data.
- **Outputs:** Domain objects conforming to core TypeScript interfaces (`CafeteriaDailyMenu`, `ShuttleRouteData`, `LibraryOccupancyReport`, `StudentClassEntry`).
- **Acceptance Criteria:** Toggling connector mode from `mock` to `live` alters data source behavior after restart with zero code modifications. Two successive runs with the same mock seed generate identical data.

5.4 AI Agent and Orchestration Intelligence

FR-13: Autonomous Multi-Tool Planning and Contextual Synthesis

- **Priority:** Must | **Verification:** T, D
- **Description:** For compound natural language queries, the AI Agent shall formulate multi-step execution plans, invoke MCP tools, and synthesize a single coherent response.
- **Inputs:** Compound student query (e.g., "*What is for lunch today, and can I catch a ring after my 10:30 class?*").
- **Processing:** The LLM inspects registered tool schemas, extracts parameters, and executes required tool calls (`get_course_schedule`, `get_cafeteria_menu`, `get_shuttle_schedule`). It matches the course end time (e.g., 12:20) against upcoming shuttle departures (e.g., 12:35) and combines the results with the dining menu.
- **Outputs:** Natural language Markdown response synthesizing data from all invoked tools, explicitly flagging any stale or simulated data sources.
- **Acceptance Criteria:** On a benchmark suite of 10 scripted compound questions (bilingual: Turkish and English), the agent correctly invokes the required tools and produces accurate answers in at least 8 out of 10 cases.

FR-14: Zero-Leakage PII Scrubbing in Agent Prompts

- **Priority:** Must | **Verification:** T, I
- **Description:** The system shall scrub student Personally Identifiable Information (PII) before transmitting context to external LLM providers.
- **Inputs:** Prompts containing student academic schedules or user metadata.
- **Processing:** In-memory regex filters detect student names, university ID numbers, and email addresses, replacing them with anonymous placeholders (e.g., `<STUDENT_ID>`, `<STUDENT_NAME>`). Raw authentication tokens are never passed to the agent context.
- **Outputs:** Sanitized prompt dispatched to the LLM API.
- **Acceptance Criteria:** Automated audit tests injecting student names and ID numbers into prompt pipelines verify that 0% of outbound HTTP requests to the Gemini API contain cleartext personal identity strings.

5.5 Web Client Presentation and UX Interaction

FR-15: Authentication State and Session Profile Management

- **Priority:** Must | **Verification:** D
- **Description:** The Next.js web application shall manage user login states via a mock SSO interface, display active student badges, and support guest access.
- **Inputs:** User login interactions and session tokens.
- **Processing:** Stores session state in secure browser cookies/local storage. Renders authenticated user headers including name and student ID badge (e.g., "Murat Can Işık - ID: 241101013"). Supports one-click sign-out.
- **Outputs:** Dynamic navigation bar reflecting authenticated or guest status.
- **Acceptance Criteria:** Upon sign-in, the student profile badge appears and the personal course timetable becomes visible. Upon sign-out, the schedule view prompts for authentication, while public widgets remain accessible.

FR-16: Client-Side Bilingual Deterministic Intent Router

- **Priority:** Must | **Verification:** T
- **Description:** The web client shall evaluate incoming queries using a lightweight rule-based router to bypass LLM inference for single-domain queries.
- **Inputs:** Raw natural language text query entered by the user.
- **Processing:** Applies regular expressions matching common Turkish and English query patterns for cafeteria, shuttle, and library lookups. If the query is single-domain and contains no conjunctions (`and`, `then`, `after`, `ve`, `sonra`, commas), it is routed directly to the tool cache. Compound, ambiguous, or unmatched queries route to the AI Agent.
- **Outputs:** Route classification: `DIRECT_WIDGET` or `LLM_AGENT`.
- **Acceptance Criteria:** On a labeled benchmark of 40 test queries (20 Turkish, 20 English), at least 90% are routed correctly. 100% of compound queries are routed to the AI Agent, ensuring zero false-positive direct routings for complex inquiries.

FR-17: Conversational Assistant View with SSE Streaming & Telemetry Badges

- **Priority:** Must | **Verification:** D, T

- **Description:** The chat view shall render real-time responses streamed over Server-Sent Events (SSE) alongside dynamic tool execution badges.
- **Inputs:** User submission in the conversational assistant input field.
- **Processing:** Listens to SSE event frames. Displays execution badges above the response area indicating active tool names, result statuses (Cache Hit, Live, Stale), and duration in milliseconds (e.g., `[Tool: get_cafeteria_menu | Cache Hit 10ms]`). Streams assistant text tokens into the view.
- **Outputs:** Interactive chat stream with collapsible telemetry chips and suggestion shortcut buttons.
- **Acceptance Criteria:** The first response characters render in the browser before generation concludes. Exactly one execution badge is displayed per invoked tool call.

FR-18: Fast-Access Campus at a Glance Dashboard

- **Priority:** Must | **Verification:** D
- **Description:** The web client shall provide an instant dashboard view displaying four core telemetry cards powered directly by cached tools without triggering LLM calls.
- **Inputs:** Dashboard navigation or manual refresh actions.
- **Processing:** Dispatches concurrent queries to tool endpoints and populates responsive card components.
- **Outputs:** Four responsive dashboard cards:
  - *Card A (Library Occupancy):* Circular capacity gauge and per-floor breakdowns.
  - *Card B (Campus Shuttles):* Next departure countdowns by route with walking-time buffer indicators.
  - *Card C (Today's Cafeteria Menu):* Tabbed Lunch/Dinner cards with calorie and allergen tags.
  - *Card D (Connector Telemetry):* Live latency badges and service health indicators for SIS, Dining, Shuttle, and Library feeds.
- **Acceptance Criteria:** All four cards load via direct tool routes without generating outbound LLM requests. Clicking any card populates the conversational input field with a related inquiry.

FR-19: Comprehensive Visual Error, Latency, and SSE Reconnection Handling

- **Priority:** Must | **Verification:** T, D
- **Description:** The client interface shall handle network latency, service outages, and connection drops gracefully.
- **Inputs:** Network disconnects, slow server responses, and connector failure payloads.
- **Processing & Outputs:**
  - *Stale Notice:* Renders an amber banner when data is stale (e.g., "*Serving cached menu from 10:15 — Dining service unavailable*").
  - *Empty State:* Renders structured notices for empty datasets (e.g., "*No classes scheduled for today*").
  - *Latency Warning:* If a query takes > 5 s, displays a "Still processing..." indicator; at 20 s, cancels the request with a retry option.
  - *SSE Auto-Reconnect:* If an SSE connection drops, retries up to 3 times with exponential backoff (1s, 2s, 4s) while displaying a reconnecting banner without clearing the user's typed input.
- **Acceptance Criteria:** Inducing simulated disconnections and service failures in mock mode confirms that each visual state (amber banner, empty state, timeout button, retry banner) renders as specified.

FR-20: PostgreSQL Telemetry Auditing and Latency Logging

- **Priority:** Should | **Verification:** T, I

- **Description:** The system shall persist operational telemetry records in PostgreSQL for auditing, monitoring, and performance analytics.
- **Inputs:** Tool execution metrics from the MCP server.
- **Processing:** Logs every tool execution with tool name, execution status, latency duration, cache hit flag, staleness flag, timestamp, and a one-way hashed student ID (plain student IDs are never logged).
- **Outputs:** Structured rows in the PostgreSQL telemetry table.
- **Acceptance Criteria:** Executing 10 tool invocations results in exactly 10 corresponding audit log rows. Inspection confirms that student identifiers are stored as irreversible cryptographic hashes.

---

6. Non-Functional Requirements (NFR)

NFR-01: Performance — Layered Latency Bounds and Decoupled Inference

- **Category:** Performance & Efficiency | **Priority:** Must | **Verification:** M
- **Specification:**
  1. Internal tool dispatching, cache lookups, and in-memory execution shall resolve with a 95th percentile (P95) latency of < 250 ms under normal load (measured across 20 simulated concurrent users over 5 minutes).
  2. Upstream live connector queries shall enforce a strict 800 ms **timeout**.
  3. External LLM inference latency shall be decoupled, tracked independently, and isolated from server-side tool execution telemetries.
- **Acceptance Criteria:** Automated Autocannon/K6 benchmark reports confirm cached P95 latency < 250 ms, with zero connector timeouts exceeding 800 ms.

NFR-02: Efficiency — Redis Cache Hit Ratio

- **Category:** Efficiency & Cost Optimization | **Priority:** Should | **Verification:** M
- **Specification:** Under normal campus usage patterns (simulated by 20 active users refreshing the dashboard every 30 seconds over a 30-minute period), the shared Redis cache hit ratio shall meet or exceed 90%.
- **Acceptance Criteria:** Telemetry analysis derived from audit logs across the 30-minute test demonstrates an aggregate cache hit ratio ≥ 90%.

NFR-03: Reliability — Fault Isolation and Cascading Failure Prevention

- **Category:** Reliability & Resilience | **Priority:** Must | **Verification:** T
- **Specification:** The total outage or network blackholing of an upstream data source (e.g., dining portal downtime) shall remain strictly isolated. A failure in one connector shall not increase the P95 latency of independent connectors by more than 10%.
- **Acceptance Criteria:** Injecting total failure into the cafeteria connector does not prevent library or shuttle tools from executing within baseline latency limits.

NFR-04: Scalability — Stateless Concurrency and Load Distribution

- **Category:** Scalability & Concurrency | **Priority:** Should | **Verification:** M

- **Specification:** The MCP server instances shall be completely stateless, persisting shared state exclusively in Redis and PostgreSQL. Two containerized server instances deployed behind the API gateway shall sustain 100 concurrent requests with an overall HTTP error rate below 1%.
- **Acceptance Criteria:** Load test execution with 100 concurrent virtual users completes with < 1% error rate. Stopping one server instance mid-test does not interrupt active sessions beyond requests in flight.

NFR-05: Security — Perimeter Token Isolation and Public TLS

- **Category:** Security & Privacy | **Priority:** Must (TLS, Secrets), Could (mTLS) | **Verification:** T, I
- **Specification:**
  1. All external web traffic shall be encrypted via HTTPS/TLS 1.3.
  2. Raw student authentication tokens and private API keys shall be loaded exclusively from environment variables, never committed to git repositories.
  3. External LLM providers shall never receive raw session tokens or unmasked PII.
- **Acceptance Criteria:** Automated repository secret scans identify zero committed credentials. Inspection confirms that outbound requests to Gemini contain only masked placeholder tokens.

NFR-06: Usability — Navigational Efficiency and Usability Standards

- **Category:** Usability & Interface Design | **Priority:** Must | **Verification:** M, D
- **Specification:** All primary campus metrics (menu, shuttles, library occupancy) shall be accessible from the dashboard within **one click** from the landing screen. In a formal usability evaluation conducted with student users, the system must demonstrate high intuitive usability and clear navigation.
- **Acceptance Criteria:** In a structured usability test conducted with at least 5 student testers:
  1. The mean System Usability Scale (SUS) score shall be ≥ 70 (indicating acceptable to good usability).
  2. At least 80% of participants shall report that the application is easy to use and intuitive in post-test assessments.
  3. 100% of primary metrics on the Campus at a Glance Dashboard are reachable in exactly one click from the initial landing view without scrolling on desktop screens.

NFR-07: Portability & Responsive Viewport Compatibility

- **Category:** Portability & UI Responsiveness | **Priority:** Must | **Verification:** D
- **Specification:** The web application shall operate across the latest stable releases of Google Chrome, Mozilla Firefox, and Apple Safari. The user interface layout shall adapt responsively across viewport widths from 375 px **(mobile phones)** to 1920 px **(desktop displays)** without introducing horizontal scrollbars.
- **Acceptance Criteria:** Visual inspection across 375 px, 768 px, and 1440 px responsive viewports confirms that all cards and chat bubbles fit within the viewport width.

NFR-08: Maintainability & Offline Test Coverage

- **Category:** Maintainability & Software Quality | **Priority:** Should | **Verification:** M, I
- **Specification:** The MCP server, connectors, and routing modules shall achieve an automated unit test line coverage of at least 70%. All test suites shall execute completely in mock mode without requiring an active internet connection.

- **Acceptance Criteria:** Running `npm test` with external network interfaces disabled executes in < 15 seconds with ≥ 70% line coverage reported by Jest.

NFR-09: Deployability — Single-Command Container Orchestration

- **Category:** Deployability & DevOps | **Priority:** Must | **Verification:** D
- **Specification:** The full multi-tier architecture (Web Client, API Gateway, MCP Server, Redis, PostgreSQL) shall start cleanly on any host system with Docker installed using a single command (`docker compose up`), referencing a documented `.env.example` template.
- **Acceptance Criteria:** A reviewer pulls the repository onto a fresh environment, executes `docker compose up`, and accesses both the web client and MCP server health endpoints without manual intervention.

---

7. Requirements Traceability Matrix (RTM)

The matrix below maps each Functional and Non-Functional Requirement to its origin in the initial Project Concept Proposal (Concept Notes), its architectural tier, its verification method, and its corresponding feature code in the Project Definition Document:

Req ID	Target Subsystem / Tier	Concept Proposal Origin	Project Definition Feature	Verification Method	Priority
FR-01	MCP Server Core / Registry	Section 3.1 - FR-1	F-1: Campus MCP Tool Server	T, D	Must
FR-02	MCP Server Transport Layer	Section 3.1 - FR-1	F-1: Campus MCP Tool Server	T, D	Must
FR-03	Connector Tier / Dining Module	Section 3.1 - FR-1 & Sec. 4	F-1 & F-7: Pluggable Connectors	T	Must
FR-04	Connector Tier / Shuttle Transit	Section 3.1 - FR-1 & Sec. 4	F-1 & F-7: Pluggable Connectors	T	Must
FR-05	Connector Tier / Library Feed	Section 3.1 - FR-1 & Sec. 4	F-1 & F-7: Pluggable Connectors	T	Must
FR-06	Connector Tier / SIS Adapter	Section 3.1 - FR-1 & FR-2	F-1 & F-5: Identity Handling	T	Must
FR-07	Fastify API Gateway / JWT	Section 3.1 - FR-2	F-5: Secure Identity Handling	T, I	Must
FR-08	API Gateway / Redis Counters	Section 3.2 - NFR-4	F-5: Secure Identity Handling	T	Must
FR-09	Redis Cache Store	Section 3.1 - FR-3 & Sec. 4	F-4: Resilient Data Access	T	Must
FR-10	MCP Core / Stale Fallback	Section 3.1 - FR-3 & Sec. 4	F-4: Resilient Data Access	T	Must
FR-11	MCP Core / Circuit Breaker	Section 3.2 - NFR-2	F-4: Resilient Data Access	T	Must
FR-12	Connector Abstract Interface	Section 3.1 - FR-5	F-7: Connectors & Mock Mode	T, I	Must
FR-13	AI Agent / MCP Client SDK	Section 3.1 - FR-6	F-3: Conversational Assistant	T, D	Must

Req ID	Target Subsystem / Tier	Concept Proposal Origin	Project Definition Feature	Verification Method	Priority
FR-14	AI Agent Prompt Sanitizer	Section 3.2 - NFR-4	F-5: Secure Identity Handling	T, I	Must
FR-15	Next.js Client Auth State	Section 5.1 - View 1	F-5: Secure Identity Handling	D	Must
FR-16	Client-Side Intent Router	Section 3.1 - FR-4 & Sec. 4	F-6: Bilingual Intent Router	T	Must
FR-17	Next.js SSE Stream Assistant	Section 5.1 - View 1	F-3: Conversational Assistant	D, T	Must
FR-18	Next.js Glance Dashboard	Section 5.2 - View 2	F-2: Campus Glance Dashboard	D	Must
FR-19	Client Error Boundaries & UX	Section 5.1, 5.2, & Sec. 7	F-2, F-3, F-4 UX Resilience	T, D	Must
FR-20	PostgreSQL Telemetry Store	Section 6.2 - Tech Stack	F-4: Resilient Data Access	T, I	Should
NFR-01	Redis & Fastify Core Engine	Section 3.2 - NFR-1	Performance Metric	M	Must
NFR-02	Redis Cache Store	Section 3.1 - FR-3	Efficiency Metric	M	Should
NFR-03	MCP Server Circuit Breaker	Section 3.2 - NFR-2	Reliability Metric	T	Must
NFR-04	Containerized Node.js Tier	Section 3.2 - NFR-3	Scalability Metric	M	Should
NFR-05	Gateway Perimeter & TLS	Section 3.2 - NFR-4	Security Metric	T, I	Must
NFR-06	UI Navigation & Cards	Section 5.1 & 5.2	Usability Metric	M, D	Must
NFR-07	Responsive Web Layout	Section 5.1 & 5.2	Portability Metric	D	Must
NFR-08	Jest & Synthetic Mock Stubs	Section 6.2 - Tech Stack	Maintainability Metric	M, I	Should
NFR-09	Docker Compose Architecture	Section 6.1 - System Arch	Deployability Metric	D	Must

Summary Totals:

- **Functional Requirements:** 20 Total (19 Must, 1 Should)
- **Non-Functional Requirements:** 9 Total (6 Must, 3 Should, 1 Could extension)
- **Compliance Status:** Fully satisfies the course criteria (minimum 10 FRs and 5 NFRs).

---

8. Revision History

Revision	Date	Lead Author	Reviewer	Summary of Changes
v0.1	22 Oct 2026	Murat Can Işık	—	Initial draft based on Concept Notes and core pseudocode.
v0.5	26 Oct 2026	Murat Can Işık	Ahmet Sefa Efendioğlu	Expanded functional requirements to 12 items; drafted preliminary NFRs.
v0.9	30 Oct 2026	Murat Can Işık	Yunus Emre Biçer	Integrated stakeholder classes, mock-mode assumptions, and acceptance criteria.
v1.0	1 Nov 2026	Murat Can Işık	Ahmet Sefa Efendioğlu	Final deliverable compiled for Assignment 1 submission; cross-audit and consistency verified.

This document constitutes the official Requirements Document deliverable for Group-ASYM in BIL 481 Software Engineering, Fall 2026, authored by Murat Can Işık and reviewed/verified by Ahmet Sefa Efendioğlu.